

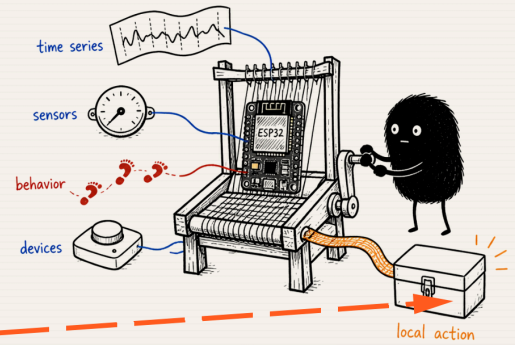
NOT GPU

NO GPU. NO FPU. NO EXCUSES.

ATTENTION

A complete, numerically validated Transformer forward pass—optimized for a **no-FPU S\$7 microcontroller**, then scaled into a wireless cluster.

PRESERVE THE MODEL. CHANGE THE EXECUTION.



80x

TOTAL CASE 1 SPEEDUP

~~2697s~~ →
33.713s

CASE 1 / EIGHT ESP32-C3 WORKERS /
64 OF 64 OUTPUTS PASS

8x

XIAO ESP32-C3
WI-FI WORKERS
PHYSICAL SETUP

NO HARDWARE FLOAT

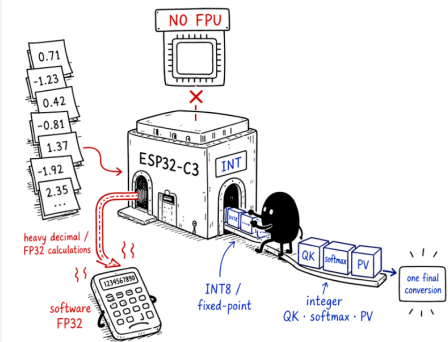
321 KB USABLE SRAM

4 MB FLASH

FULL TRANSFORMER BODY

01 / NUMERIC PATH

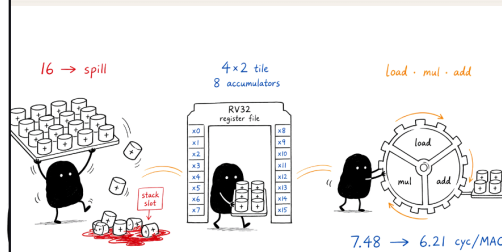
MAKE FLOAT DISAPPEAR.



Q15 ACTIVATIONS · Q12 WEIGHTS
513-ENTRY EXP LUT · STABLE SOFTMAX

02 / MACHINE PATH

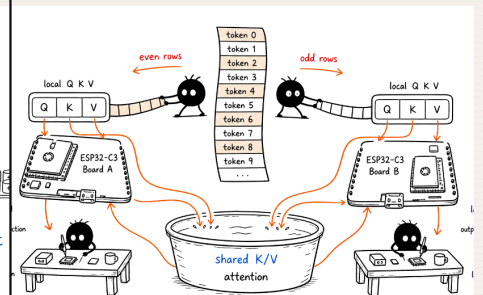
MAKE EVERY REGISTER COUNT.



4x2 GEMM TILE · 8 ACCUMULATORS
HAND-SCHEDULED MAC CHAINS · FUSED RESIDUALS

03 / CLUSTER PATH

MAKE ONE BOARD BECOME EIGHT.



PARITY-INTERLEAVED TOKEN ROWS
OVERLAP K/V EXCHANGE WITH COMPUTE

SCALE OUT, STAY HONEST.

PHYSICAL MEASUREMENTS.
SAME TIMING SCOPE.

8x

ESP32-C3 BOARDS
WORKING IN PARALLEL

8.00x

EIGHT WI-FI WORKERS VS.
ONE IDENTICAL WORKER

64/64

CASE 1 OUTPUTS
PASSING THE OFFICIAL GATE

NUMERICALLY VALIDATED

TinyCluster — Karthik Gangula · Mingchen Yang · Yuma Ochi

Complete attention · LayerNorm · GELU · FFN · residuals · local execution

OPEN SOURCE ↗

github.com/chizuchizu/techjam2026

A Transformer on an S\$7 microcontroller

The benchmark asks you to make a Transformer layer fast on a GPU. We asked the opposite question — what is the *smallest* machine that can run it at all — and answered it on a 160 MHz RISC-V chip with no floating-point unit and 321 KB of RAM. One forward pass went from 42.15 seconds to 1.996. Then we built a cluster.

Sseed XIAO ESP32-C3 · 10 of 14 official cases measured on physical hardware · every result gated per element against the PyTorch reference

CASE-2 FORWARD	AFTER 24 STEPS	TWO BOARDS	ACCURACY
42.15 s	1.996 s	1.276 s	25/25
Where we started. Measured, one board.	21.1x faster, same board, same weights.	33.0x against the original baseline.	Device seeds, zero failing elements.

01

THE PREMISE

Why point a GPU benchmark at a microcontroller

The official brief is unambiguous: optimise a fixed Transformer layer *on a given GPU*, submit GPU kernels, and report the speedup against a PyTorch baseline. We did not do that. It is worth saying so plainly before anything else in this report is read.

What we did instead was take the same fourteen shapes, the same reference implementation, and the same per-element accuracy gate, and ask a question the brief does not ask: **how far down the hardware ladder does this workload still run?** Not on a smaller GPU — on a microcontroller costing about seven Singapore dollars, with no floating-point unit, no DRAM, no operating system worth the name, and less memory than a single one of the benchmark's activation tensors.

The reason this is interesting is that a GPU makes the interesting constraints invisible. On an H200 the case-2 forward runs at under 1% model-FLOP utilisation and nobody notices, because it finishes in milliseconds either way. Move the same arithmetic onto a part with a 160 MHz single-issue in-order core and every decision that a GPU absorbs — where a tensor lives, what numeric format it carries, how many times a weight is read from flash — becomes measurable, and usually dominant. The workload stops being a throughput problem and becomes a *memory and numerics* problem.

STATED HONESTLY

This is a re-framing of the brief, not a fulfilment of it. We are not claiming a GPU kernel result. What we claim is a reproducible study of the same benchmark on hardware roughly twelve million times slower in peak arithmetic, where the answers are different in kind rather than in degree — and where 10 of the 14 official shapes still pass the gate.

There is a second reason, and it is the practical one. A cluster of eight of these boards costs about \$56 and draws a couple of watts. If a Transformer of this size can be made to run there, inference stops needing a datacentre for a whole class of embedded problems. That is a claim the benchmark cannot make on a GPU, because the GPU was never the constraint.

What an ESP32-C3 actually is

Everything in this report follows from four lines of a datasheet, so they are worth stating precisely before the engineering starts.

CORE	FLOATING POINT	USABLE RAM	FLASH
160 MHz Single RV32IMC core. In-order, single-issue, 4-stage.	none No FPU. Every fp32 operation is a library call.	321 KB 321,296 B of dram0_0_seg. No PSRAM, no DRAM.	4 MB 3.1 MB app partition, read through a 16 KB cache.

The board is a Seeed XIAO ESP32-C3, about the size of a thumbnail and about five dollars. There is no data cache — the 16 KB cache covers execute-in-place reads from flash only. IRAM and DRAM are aliases of the same physical SRAM pool, which matters later: you cannot buy instruction-memory speed by spending data memory, because there is only one pool.

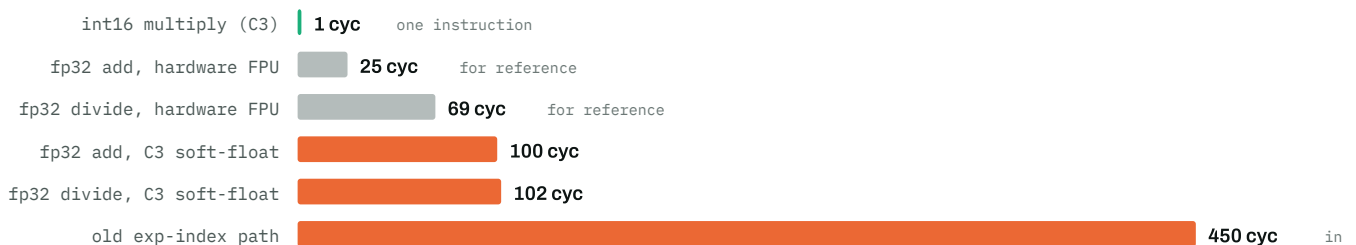
For scale: the case-2 forward is 0.134 GFLOP of arithmetic over 398,592 parameters. At fp32 those weights are 1,594,368 bytes — they fit in flash with room to spare, and do not fit in RAM at all, four times over. That single fact shapes the entire implementation.

03

CHALLENGE ONE

There is no floating-point unit

The Transformer reference is written in fp32. The chip cannot do fp32. Every add, multiply, divide and exponential in the reference becomes a call into libgcc's soft-float library, and those calls are not cheap in the way a cache miss is expensive — they are cheap operations that have simply become a hundred times slower.



The soft-float tax. Cycles per operation. A part with a hardware FPU pays 25 cycles for an fp32 add; the C3 pays about 100, because the operation is synthesised in software. The integer multiply the chip *can* do costs one. This ratio — roughly 100:1 — is why the entire optimisation effort is a migration from floating point to fixed point, and not a search for better cache behaviour. Figures from Espressif's own performance guidance, recorded in the project's optimisation log.

Profiling the untouched implementation made the consequence concrete. Of the 42.15 seconds a single forward took, **30.09 seconds — 71.4% — was attention**, and essentially all of it was soft-float: 16.7 million fp32 dequantise-multiplies in the QK dot products, about a million expf calls in the softmax, and 8.4 million fp32 multiply-accumulates in the PV product.



attention Q/K/V projections other GEMMs GELU quantise LayerNorm + residual

Where the 42.15 seconds went. Attention was not the bottleneck in the usual sense of being the largest matrix multiply — the GEMMs move far more data. It dominated because it was the part still doing floating-point arithmetic element by element.

The GEMMs were already running on an integer path and were still slow for a different reason: measured at **36 cycles per multiply-accumulate** where an ideal int32 MAC loop on

this core should cost 5–8. That is a code-generation and register-pressure problem, not a numerics one, and it needed a different fix.

04

CHALLENGE TWO

The memory is smaller than one tensor

The linker grants the application 321,296 bytes. A single 128x128 fp32 activation is 65,536 bytes. The reference implementation wants several of them live at once, plus per-head projections, plus quantisation scratch — and the framework, the WiFi stack and the stack itself all come out of the same pool.



321,296 B dram0_0_seg — read from the link map, not assumed

■ activation buffers ■ Q15 scratch ■ per-head Q/K/V ■ framework, drivers ■ initialised data

The whole budget, one bar. The optimised case-2 firmware: three 64 KB activation buffers, a 32 KB Q15 scratch, three per-head buffers, and roughly 13 KB of framework state, leaving about 47 KB. Earlier in the optimisation sequence one build sat at **99.9% of the segment with 384 bytes free**. Weights are not on this chart at all — all 1.59 MB of them live in flash and are streamed.

Three consequences run through the whole project.

Weights never enter RAM. They are embedded in the flash image and read through the 16 KB execute-in-place cache as the kernels stream them: 786,432 bytes of int16 weight matrices per forward, derived from measured call counts. Any scheme that required a resident weight matrix was off the table from the start.

Buffers are aliased, not allocated. The final layout re-uses the same 64 KB regions under different types at different points in the forward — the attention context is an int16 view of a buffer that held fp32 a moment earlier. Adding a buffer was almost never an option; the residual path was eventually carried as int32 inside a union with the fp32 view precisely so it would cost zero new bytes.

Some shapes simply do not fit. Case 9 (H=1) and case 10 (H=2) failed to link — the linker reported dram0_0_seg overflowed by 73,072 bytes and by 23,920 bytes respectively. Per-head state scales with S·HD, so fewer heads means larger per-head buffers. Both were eventually made to fit by patching the SDK's own static allocations, which is discussed in section 8.

THE CONSTRAINT THAT BIT HARDEST

Adding WiFi to a compute node costs roughly 85,500 B of static DRAM plus about 69 KB of heap. The full-sequence firmware needs 274,564 B. Together that is about 429 KB against a 328 KB budget — short by roughly 100 KB. A board could compute, or it could talk, but not both, until the schedule was rewritten.

05

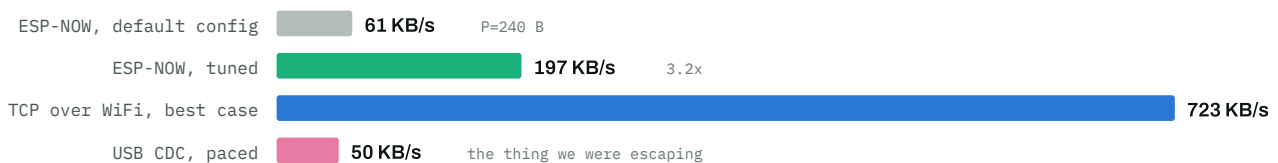
CHALLENGE THREE

Talking between boards costs more than computing

If one board is too small, use several. But the moment a forward is split across boards, the parts have to exchange activations — and on this hardware the link is slower than the arithmetic it is meant to accelerate.

The numbers set the scale. A case-2 forward has to move 64 KB in and 64 KB out. Over USB CDC, paced to the rate the chip's native USB will accept without dropping bytes, that is about **1.3 seconds per input** — comfortably longer than the 1.996 second forward it feeds. Transport was 51% of end-to-end time in the two-board batch runs.

So the radio had to be made fast. We built a dedicated link benchmark to find out what an ESP32-C3 pair can actually sustain, and the answer depended almost entirely on configuration rather than on protocol design.



Link throughput, measured. Tuning ESP-NOW — disabling power save, unicasting to a paired peer, forcing MCS7 with HT40, raising TX power — took it from 60.9 to 197.0 KB/s at a 240-byte payload, with median round-trip latency falling from 1,915 μ s to 850. The best raw figure is TCP at a 4 KB payload (5.787 Mbps), but TCP's window on this stack caps a single connection well below what the PHY allows.

Even at 197 KB/s, a naive split loses. The first working two-board case-2 build used blocking TCP and spent **1.83 seconds of a 4.64-second forward** waiting on the link. Streaming it per attention head over TCP made it worse — 2.57 to 3.38 seconds blocked — because the per-transfer overhead multiplied.

What worked was UDP with negative-acknowledgement recovery, streamed per head, on a dedicated link task, so that a board releases each head's K/V block the instant it is projected and only blocks when attention actually needs the peer's. That took time spent waiting from 1.83 seconds down to **5–88 milliseconds** against roughly 1.6 seconds of raw transfer — the transfer did not get faster, it got hidden.

LINK STRATEGY	WALL TIME	BLOCKED ON LINK
blocking TCP, whole 32 KB block	4.64 s	1.83 s
TCP streamed per head, overlapped	5.87–6.62 s	2.57–3.38 s
UDP + NAK, streamed per head	3.06 s	0.003–0.032 s

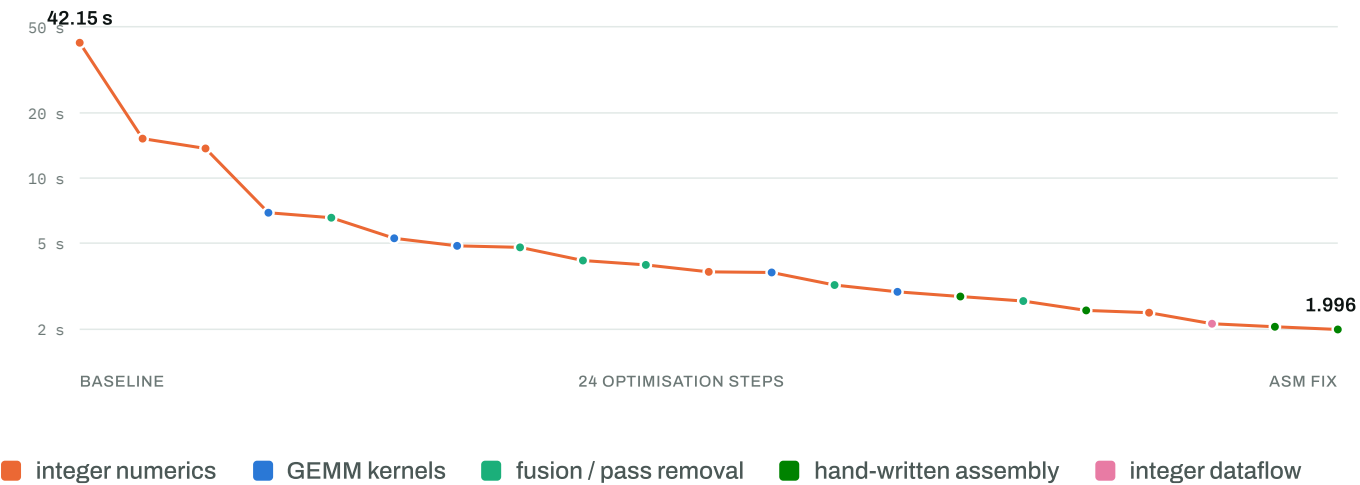
Measured on the same two boards on opt18-era kernels; the wall times predate the final kernel work, so compare the columns to each other rather than to the headline figure.

06

THE METHOD

Twenty-four steps from 42.15 to 1.996 seconds

Case 2 — one input, 128 tokens, 128 dimensions, 4 heads, 4 layers — is the shape everything else was built on. It is the only case whose baseline is a real measurement rather than a projection, and it is where every technique was developed and gated before being applied elsewhere.



The ladder. Seconds per forward, log scale, one point per recorded optimisation. There is no single win: the largest step (integer attention) accounts for about 64% of the total gain, and the remaining 36% is twenty-three steps of three to eleven percent each. Point colour marks the family of change.

The work falls into five families, and they are worth separating because they attack genuinely different costs.

1. Move attention into fixed point

The largest single win, and the obvious one once the profile existed. QK dot products became int64 accumulations with the dequantisation scale folded into a single fp32 multiply per logit. The softmax exponential became a 513-entry int16 lookup table with 7-bit linear interpolation — and then, one step later, an all-integer table index: a single Q32 fixed-point multiply replacing an int64→float conversion, two fp32 multiplies and a float→int conversion, about 450 cycles reduced to one multiply. **That step alone took the exp stage from 194.8 to 23.2 μ s per call.** The PV product went last: rescaling each softmax row so it sums to approximately 32,767 keeps the accumulation inside int32, so each element costs one 16x16 multiply with no 64-bit accumulate.

2. Rewrite the GEMM inner loop, six times

Measured at 36 cycles per MAC at the start. The progression — core through core5 — is entirely about register tiling: reuse each fetched weight across more output rows, then reuse each activation across more output columns, then hoist loads across a K-pair to hide flash latency. The instructive failure is core4_v2, which needed roughly 36 live registers against the RV32IMC ABI's 28 and spilled on every iteration. core5 uses a smaller 4x2 tile, fits in about 20 registers, and runs faster despite doing less per iteration: **7.48 to 6.21 cycles per MAC.** Every kernel in the chain is bit-identical to its predecessor, because the accumulation order never changes.

3. Delete passes rather than speed them up

The recurring discovery was that the scaffolding around the arithmetic cost more than the arithmetic. Separate amax-scan, quantise and staging passes over 64 KB buffers were removed one at a time by fusing them into the producer: GELU folded into the FFN quantisation; LayerNorm's output scan replaced by an *analytic* bound on the maximum, so the scale is known before normalising rather than after; attention writing Q15 context directly so the output projection's quantise step disappears entirely.

4. Hand-write the hot loops in assembly

Once the C kernels were at the register limit, the remaining wins were scheduling ones the compiler would not make. The head-GEMM inner loop became 28 instructions per 8 MACs with a read-after-write distance of 8 — far enough that the multiplier never stalls. A C fallback is kept for every assembly path so the host tests still build.

5. Make the dataflow integer end to end

The last floating-point object was the residual itself, and it was forcing three costs per layer: float GEMM epilogues, separate residual-add passes, and a float quantise inside every LayerNorm. Carrying it as int32 at a fixed scale — inside a union with the fp32 view, so it cost no new memory — let the GEMMs accumulate straight into the residual at about **10 instructions per element instead of 268**, and deleted the residual passes entirely. Accuracy improved.



■ Q/K/V projections ■ other GEMMs ■ attention ■ LayerNorm + final ■ residual ■ GELU

Where the time goes now. The same forward, profiled again. Attention has fallen from 71% of the total to about 19%, and the GEMMs — which were always doing most of the arithmetic — are now correctly the largest cost. This is what a successful optimisation looks like in a profile: not everything smaller, but the bottleneck moved to where the real work is.

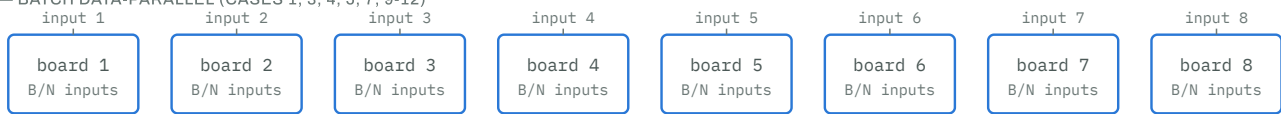
The cumulative effect on utilisation is the cleanest summary: model-FLOP utilisation went from **2.0% to 42.2%** of the board's derived peak, and held within 0.1 percentage points across cases 1 through 5.

► Things that did not work — and why they are in this report

Two ways to split, and why they scale differently

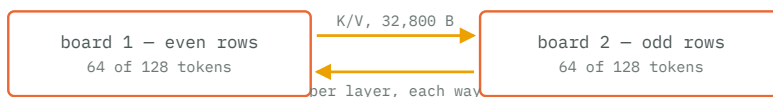
Thirteen of the fourteen cases have a batch dimension greater than one. That single fact splits the parallelisation problem cleanly in two, and the two halves behave nothing alike.

A — BATCH DATA-PARALLEL (CASES 1, 3, 4, 5, 7, 9-12)



Boards exchange nothing. Speedup is exactly $\min(B, N)$.

B — TOKEN-ROW SPLIT, INSIDE ONE FORWARD (CASE 2, $B=1$)



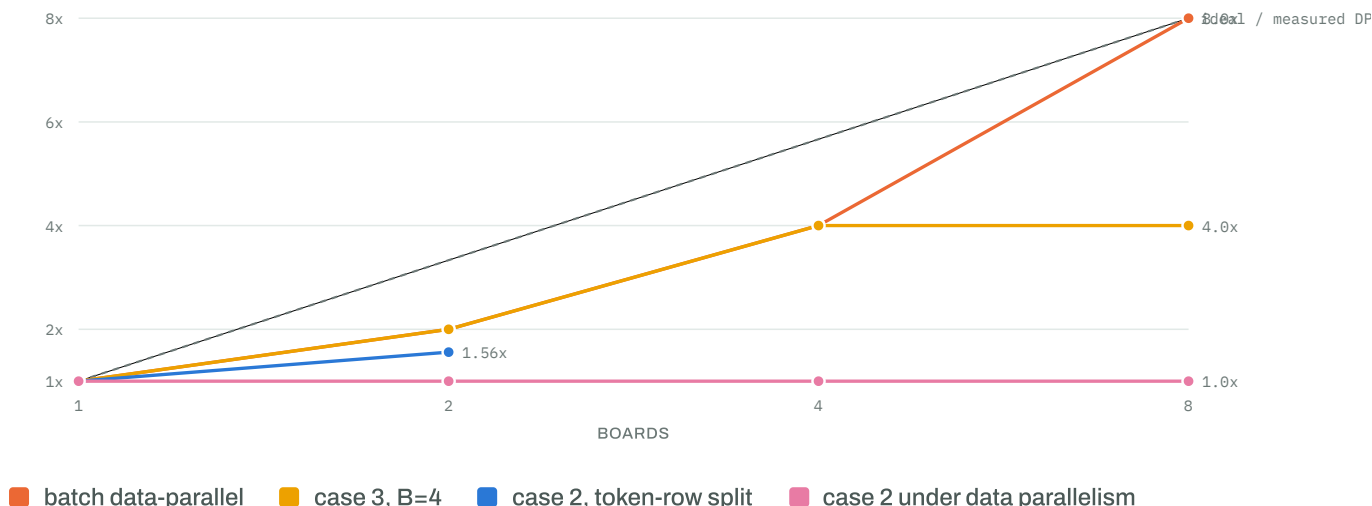
Causal attention needs $j \leq i$, so every layer forces one K/V exchange: 131,200 B per forward.

The two shapes. When $B > 1$ the inputs are independent forwards over shared weights, so board i takes input $i \bmod N$ and no board ever speaks to another. When $B = 1$ there is only one forward to split, and it has to be cut *inside* the model — which causal attention makes expensive.

The batch case is as good as parallelism gets. The boards share no state, exchange nothing during a forward, and the speedup is exactly $\min(B, N)$. Measured on eight physical boards over WiFi: **8.00x** for cases 1, 4 and 5, with 213 of 213 forwards passing the gate and zero failing elements. Case 3 saturates at 4.00x because $B=4$ — four boards work and four sit idle, which we report as 4.00x rather than counting the idle boards.

Case 2 is the interesting one, because $B=1$ means data parallelism has nothing to divide. The only remaining axis is the token dimension. Every operator in the case-2 body is per-token — LayerNorm, all four projections, both residuals, the FFN, GELU, the final norm — *except* causal attention, which needs every key and value at or before the current row. So each board takes half the token rows and the boards exchange one K/V block per layer.

A detail that matters more than it looks: rows are assigned by parity, not by cutting the sequence in half. Under a causal mask, row i does $i+1$ units of work, so a contiguous split would give one board 2,080 score rows and the other 6,176. The parity interleave gives 4,096 against 4,160 — **a 1.6% imbalance instead of a 3x one**.



Measured scaling. Batch-parallel cases track the ideal line exactly, because there is nothing to synchronise. Case 3 stops at 4x because it runs out of batch. Case 2 gets 1.56x on two boards and cannot use data parallelism at all — its single input activates exactly one node no matter how many are available.

Why 1.56x and not 2.00x

This is the most honest number in the project, so it is worth explaining rather than rounding up. Three costs do not halve when the tokens do:

- **Weight traffic is replicated.** Both boards stream all 24 quantised weight matrices from flash every layer. That cost is per board, not per row, so splitting the rows does not touch it.
- **Per-call fixed costs double.** Every GEMM, LayerNorm and quantise is now invoked twice over half the rows each, so the amax scans, the per-layer GELU table rebuild and the scale arithmetic are all paid twice.
- **Attention keeps a 1.6% imbalance,** plus the extra per-source logit conversion in the merged softmax.

There is a further wrinkle worth recording: the same split measured **1.73x on earlier kernels**. The partition never changed — the single-board baseline got 2.6x faster underneath it, so the fixed costs became a larger fraction of a smaller number. A parallel speedup is a ratio, and optimising the denominator makes it look worse.

08

COVERAGE

Ten of fourteen shapes, on physical hardware

Case 2 was the development shape. The question that decides whether any of this generalises is what happens when the geometry changes — and the answer was mostly determined by memory, not by arithmetic.

CASE	SHAPE (B,S,D,H,F,L)	BASELINE 1 BOARD	OPTIMISED 1 BOARD	2 BOARDS	8-NODE WIFI	STATUS	GATE
01	64,128,128,4,128,4	2,697.6 s EST	127.36 s	63.7 s	33.713 s	MEASURED	64/64 PASS
02	1,128,128,4,128,4	42.15 s	1.990 s	1.276 s	4.220 s †	MEASURED	25/25 seeds
03	4,128,128,4,128,4	168.6 s EST	7.96 s	4.0 s	4.218 s ‡	MEASURED	4/4 PASS
04	16,128,128,4,128,4	674.4 s EST	31.84 s	15.9 s	8.438 s	MEASURED	16/16 PASS
05	128,128,128,4,128,4	5,395.2 s EST	254.72 s	127.4 s	67.451 s	MEASURED	128/128 PASS
06	10000,128,128,4,128,4	—	—	—	—	PARTIAL	builds, host gate PASS
07	64,128,32,4,32,4	—	30.427 s	15.822 s	3.963 s	MEASURED	64/64 PASS
08	64,128,1024,4,1024,4	—	—	—	—	NOT RUN	weights exceed flash
09	64,128,128,1,128,4	—	138.027 s	—	28.508 s	MEASURED	64/64 PASS
10	64,128,128,2,128,4	—	138.536 s	119.101 s	29.793 s	MEASURED	64/64 PASS
11	64,128,128,16,128,4	—	138.610 s	206.354 s	51.604 s	MEASURED	64/64 PASS
12	64,32,128,4,128,4	—	33.879 s	17.091 s	4.282 s	MEASURED	64/64 PASS
13	64,1024,128,4,128,4	—	—	—	—	NOT RUN	activations exceed SRAM
14	32,100000,1024,16,1024,2	—	—	—	—	NOT RUN	impossible by ~4 orders

All times are the device's own measurement of the complete four-layer body, excluding host serial transfer. † case 2 has B=1, so only one data-parallel node activates — this is the tiled single-forward time, not an eight-board speedup. ‡ case 3 has B=4 and saturates at four active nodes. **Only case 2's baseline is a measurement**; cases 1, 3, 4 and 5 were never run on the pre-optimisation firmware, so their baselines are projected as B x 42.15 s and marked accordingly.

What porting actually took

The kernels were made shape-parametric, so cases 7 and 12 — which only shrink D, F or S — needed no new numerics at all. Case 7 (D=F=32) is small enough that its WiFi image links at 108,300 B of static RAM with no tiling required.

The head-count cases were harder, and in an unintuitive direction: **fewer heads is worse**. Per-head state scales with S·HD, so at H=1 the per-head buffers are eight times larger than at H=8. Cases 9 (H=1) and 10 (H=2) did not link — over by 73,072 and 23,920 bytes. Making them run meant going below our own code and shrinking the SDK's static allocations: the interrupt stack, the coredump stack, the process-status buffer. Case 9 now links at 273,180 B and runs; case 10 needed a buffer overlay and sits at 97.25% of the segment with 8,848 bytes to spare.

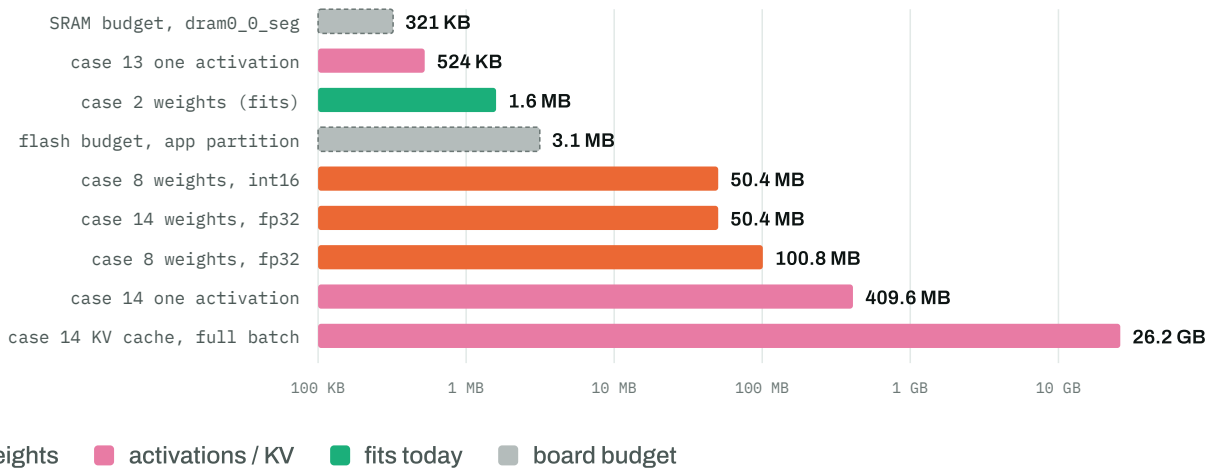
Case 11 (H=16) fit comfortably and is the counterexample that confirms the mechanism — more heads means smaller per-head state, 256,180 B and 78.2% of budget.

A RESULT WE DID NOT WANT

To fit WiFi alongside the model, a 16-row sequential tile schedule was added. It works — but a tiled forward takes **4.215 s against 1.990 s** on the untiled USB build. Tiling is a memory enabler, not a speed optimisation, and it costs 2.12x per device. For case 11 the consequence is blunt: two tiled replicas are *1.49x slower* than one plain board, and four only just cross over at 1.34x.

Four cases we cannot run, and exactly why

Three of the four failures are memory, and the arithmetic is not close. It is worth doing the sums explicitly rather than saying "too big", because the margin tells you whether more boards would fix it.



What the hard cases need against what one board has. Log scale — the dashed bars are the budgets. Case 8's fp32 weights are 32x the flash partition, and still 16x over when quantised to int16. Case 14's single activation tensor is 1,250x the entire SRAM, and its KV cache across the batch is roughly 24 GiB.

Case 6 — (10000, 128, 128, 4, 128, 4). Not a memory problem at all.

The firmware builds, links at 83.2% of RAM, and passes the host gate 50/50. The blocker is time: 10,000 forwards at 1.990 s is **19,900 seconds — about 5.5 hours** of device compute, plus roughly 1.31 GB of serial transfer to feed it. We did not run it, and we do not claim it. With enough boards it is straightforwardly feasible; the project's own estimate is around 300.

Case 8 — (64, 128, 1024, 4, 1024, 4). The weights do not fit in flash.

D=F=1024 gives 25,208,832 parameters — **100,835,328 bytes in fp32, 32.1x the 3.1 MB app partition**. Quantising to int16 halves it and leaves it 16x over. A *single* D×D projection matrix in int16 is 2 MiB, which is 6.5x the entire RAM budget, so no one weight matrix is ever resident. Compute is not the issue: 429.5 GFLOP would take about 1.8 hours at the measured rate. This one needs weight sharding across roughly 28–32 boards.

Case 13 — (64, 1024, 128, 4, 128, 4). The weights fit; the activations do not.

The weights are identical to case 2's and sit happily in flash. S=1024 is what breaks it: one S×D fp32 activation is **524,288 bytes against 321,296 of RAM** — a single tensor exceeds the whole budget before anything else is allocated. The dense attention score matrix would be 4 MiB per head, 13x the segment. This is the case that genuinely needs online, block-wise attention that never materialises S×S, which we designed but did not implement.

Case 14 — (32, 100000, 1024, 16, 1024, 2). Out by four orders of magnitude.

One S×D fp32 activation is 409,600,000 bytes — **1,250x the SRAM**, and about 100x the entire flash part. The KV cache across the batch is roughly 24 GiB. The dense score matrix for one head of one layer would be 40 GB. The arithmetic is 2.70 PFLOP, which at the measured single-board rate is **464 days**. Storage alone would need something over 5,600 boards. This case is not a scaling problem; it is a different machine.

10

EVIDENCE

How we know any of this is true

Two claims in this report are load-bearing — that it is fast, and that it is still correct — and the second is the one that makes the first mean anything.

The gate

The benchmark's own criterion, applied per output element: $|\text{error}| \leq 0.002$ or $|\text{error}| \leq 0.02 \times |\text{reference}|$. Note the disjunction — it is not `torch.isclose`. The official harness runs 5 random trials; we run **25 seeded trials and require zero failing elements**, on the device and again on the host, in both the integer FAST path and a pure fp32 EXACT path kept bit-identical throughout as a reference.

Across every case the worst device error observed was **1.46e-3 against a 2e-3 tolerance** — a margin of about 1.4x maintained through 24 numeric changes. Many individual steps are stronger than the gate: six of the kernel rewrites are *bit-exact* against their predecessors, verified element by element rather than statistically.

One thing worth being clear about: the benchmark ships no dataset and no trained weights. The model is built from seeded random values, so "correct" means *matching the PyTorch reference on those same random weights*. There is no accuracy claim about a task here, only an equivalence claim about arithmetic.

The profiler

Halfway through, the measurements themselves became the weak link. The per-op timings in the optimisation log were transcribed by hand from serial output; every memory figure was copied out of the build summary; the flash-traffic number was arithmetic on assumed call counts. So we built **tinyprof**, an operator-level profiler for this board, and pointed it at both firmwares.

It found three things immediately. Zones were timed with a 1 μs timer and discarded when they rounded to zero, so the residual-add op was *absent from every profile* and its sibling reported 2 calls out of 12 — fixed by timing with the cycle counter at 6.25 ns. Two ops were nested inside others and being double-counted in the rankings. And a 32 KB scratch buffer was missing from the memory census, caught by cross-checking the firmware's own declaration against the linked ELF.

It also reproduces the previously hand-computed 768 KiB/forward of flash traffic from measured call counts — and finds 9,216 bytes of LayerNorm parameter reads the hand calculation had missed.

WHAT WE WOULD FLAG TO A REVIEWER

- **Four of five baselines are projections.** Only case 2's 42.15 s was measured on pre-optimisation firmware. Cases 1, 3, 4 and 5 are B x 42.15 s and marked as such everywhere.
- **The 8.00x node scaling costs per-device efficiency.** Tiled WiFi nodes run at 19.9% utilisation against 42.2% for the best single board, so against that board the true eight-node gain is 3.78x, **not 8.00x**. Both numbers are real; only one of them is the honest headline.
- **Not every port was a win.** Case 12's optimised build measured 0.529 s against an earlier 0.493 s — about 7% slower. It is in the table at its measured value.
- **The peak used for utilisation is derived, not from a datasheet.** 160 MFLOP/s per board is our own bound. Counting only unmasked causal attention rather than dense S^2 would scale every utilisation figure down about 13%.
- **The radio was never shown to steal compute.** Per-forward time is constant regardless of node count, and the documented cost of WiFi is memory. But channel contention across eight nodes was not measured.

Hardware: Seeed XIAO ESP32-C3, RV32IMC at 160 MHz, no FPU, 400 KB SRAM (321,296 B usable), 4 MB flash. Toolchain: PlatformIO with the Arduino-ESP32 core on ESP-IDF 4.4. Reference: the competition's own `torch_transformer_benchmark.py`, seed 1234. Every figure in this report is either a physical device measurement or explicitly labelled as a projection or a derived bound. Per-case evidence, raw captures and the optimisation log live in the repository under `benchmarks/case-NN/`.